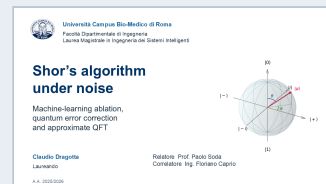


Discorso di laurea · 15 minuti

Leggi con calma, come se lo raccontassi. La barra azzurra con il triangolo indica quando cambiare slide; a destra vedi la slide che deve comparire. Durante la demo i tempi tra parentesi sono quelli del video.

► INIZIO · 1

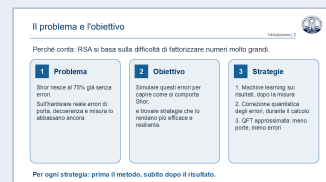
Copertina



Buongiorno. Ringrazio il presidente e tutta la commissione per la parola. Sono Claudio Dragotta e vi presento la mia tesi, che ho svolto con il professor Soda e l'ingegner Caprio.

► CAMBIA SLIDE · 2

Il problema e l'obiettivo



La sicurezza di RSA, che protegge buona parte delle nostre comunicazioni, si basa sul fatto che fattorizzare un numero molto grande è difficilissimo per un computer classico. L'algoritmo di Shor, su un computer quantistico, lo farebbe in tempo polinomiale.

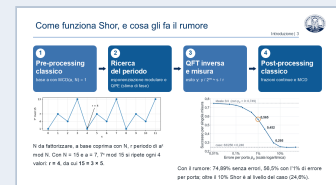
Ma Shor non è certo nemmeno in condizioni ideali: trova i fattori circa tre volte su quattro, il 75%. Sull'hardware quantistico reale si aggiungono poi gli errori: errori di porta, decoerenza, errori di misura.

Ho quindi simulato questi errori per capire come si comporta l'algoritmo, con un obiettivo preciso: trovare strategie che lo rendano più efficace e più resiliente.

Ho studiato tre strategie: il machine learning applicato ai risultati, la correzione quantistica degli errori e la QFT approssimata.

► CAMBIA SLIDE · 3

Come funziona Shor, e cosa gli fa il rumore



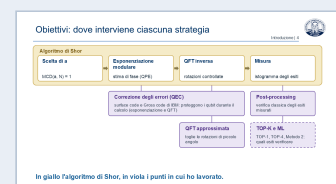
Prima vediamo come funziona Shor. Abbiamo N , il numero da fattorizzare, e una base a che non ha divisori in comune con N . La funzione a alla x modulo N è periodica, e il compito del computer quantistico è trovarne il periodo r . Nel mio esempio N è 15 e a è 7: nel grafico a sinistra i valori si ripetono ogni quattro passi, quindi r è 4, e da r un calcolo classico ricava 15 uguale 3 per 5. Il circuito usa 12 qubit: 8 di controllo e 4 di lavoro.

Il grafico a destra mostra cosa gli fa il rumore. Dopo ogni porta inserisco con una certa probabilità un operatore di Pauli casuale, cioè un'inversione del bit, della fase o di entrambi. In alto c'è il 75% ideale; in basso, tratteggiato, circa il 25%, cioè quello che otterrei con risultati del tutto casuali.

Senza rumore ottengo il 74,89%, come previsto. Basta l'1% di errore per porta per scendere al 56%, e oltre il 10% Shor è al livello del caso: non funziona più. Da qui la domanda della tesi: come renderlo più resiliente?

► CAMBIA SLIDE · 4

Obiettivi: dove interviene ciascuna strategia



Ho provato tre strategie, che agiscono in punti diversi dell'algoritmo. In giallo ci sono i passi di Shor: scelta di a , esponenziazione modulare, QFT inversa e misura; in viola i punti in cui ho lavorato.

La prima agisce dopo la misura: è il post-processing, e qui ho usato il machine learning per capire quali risultati sono buoni.

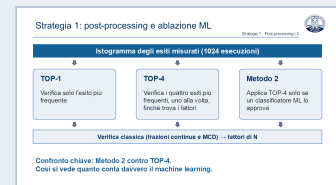
La seconda è la correzione degli errori, che protegge i qubit durante il calcolo. L'ho studiata in esperimenti separati, confrontando codici e decoder.

La terza è la QFT approssimata, che toglie porte alla QFT inversa.

Per ognuna vi mostro prima il metodo e subito dopo il risultato.

► CAMBIA SLIDE · 5

Strategia 1: post-processing e ablazione ML



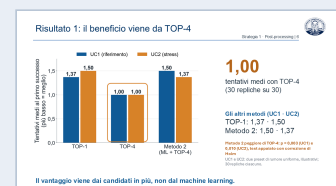
Prima strategia: il post-processing. Ogni esecuzione di Shor dà un istogramma di risultati, e bisogna decidere quali verificare.

TOP-1 verifica solo il più frequente. TOP-4 verifica, uno alla volta, i quattro più frequenti, finché trova i fattori. Il terzo metodo, che nella tesi chiamo Metodo 2, applica TOP-4 solo se un classificatore di machine learning lo approva: il classificatore deve riconoscere i risultati buoni da quelli non buoni. La verifica finale è sempre lo stesso calcolo classico.

Il rischio è dare al machine learning un merito che viene invece dal provare più candidati. Per questo il confronto decisivo è tra il Metodo 2 e TOP-4.

► CAMBIA SLIDE · 6

Risultato 1: il beneficio viene da TOP-4



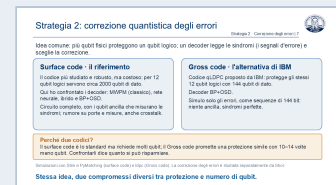
Ed ecco il risultato. Barre più basse significano meno tentativi per trovare i fattori. Nel riquadro arancione, TOP-4 trova i fattori sempre al primo tentativo, in tutte le 30 repliche.

Il classificatore è molto accurato, eppure il Metodo 2 non abbassa le barre, anzi fa peggio di TOP-4, perché a volte scarta proprio i risultati buoni. In pratica il classificatore premia gli stessi esiti più frequenti che TOP-4 prova già: non mi dà un'informazione in più. C'è anche un limite da dichiarare: l'ho addestrato su dati simulati, non su un computer quantistico reale.

Quindi il vantaggio viene dal provare più candidati, non dal machine learning.

► CAMBIA SLIDE · 7

Strategia 2: correzione quantistica degli errori



Seconda strategia: la correzione quantistica degli errori. Più qubit fisici proteggono un solo qubit logico: i controlli segnalano dove c'è un errore e un decoder decide come correggerlo.

Ho usato due codici. Il surface code è il riferimento, robusto ma costoso: per 12 qubit logici servono circa 2000 qubit di dato. Qui simulo il circuito completo, con i qubit ancilla che misurano le sindromi a ogni ciclo, e ho confrontato i decoder: quello classico, MWPM, una rete neurale, un ibrido e BP+OSD.

Il Gross code, proposto da IBM, promette una protezione simile con da 10 a 14 volte meno qubit. Qui invece simulo solo gli errori, come sequenze di 144 bit sui qubit di dato, senza ancilla e con sindromi perfette: è un modello più semplice.

► CAMBIA SLIDE · 8

Risultato 2: la QEC aiuta sotto soglia

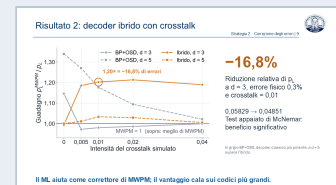


Il primo risultato è la soglia. Ogni colore è un codice più grande. Nella zona verde, sotto soglia, le curve dei codici grandi stanno più in basso: fanno meno errori. Nella zona rossa l'ordine si inverte e aggiungere qubit peggiora le cose. La soglia è il punto in cui le curve si incrociano, nel cerchio arancione: l'ho stimata tra lo 0,8 e lo 0,9% di errore.

Cosa significa per Shor? Torno alla curva del rumore: se la correzione porta l'errore residuo per porta allo 0,2%, Shor resta intorno al 70%; allo 0,5% resta al 64%; senza correzione, all'1%, scende al 56%. Quindi la correzione funziona, ma non riporta Shor al 75%. Preciso che è una stima: ho simulato Shor con un errore residuo per porta, non l'intero circuito corretto con i suoi qubit ausiliari.

► CAMBIA SLIDE · 9

Risultato 2: decoder ibrido con crosstalk

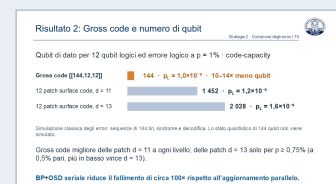


Qui il machine learning trova un ruolo. Sopra la linea dell'1 il decoder sbaglia meno di quello classico. Senza crosstalk la rete da sola non batte il decoder classico. Con il crosstalk, cioè errori correlati tra qubit vicini, il decoder ibrido, la linea arancione continua, sta sopra: nel punto cerchiato gli errori scendono del 16,8%.

Sui codici più grandi, le linee tratteggiate, il vantaggio quasi sparisce, e BP+OSD, in grigio, a distanza 5 fa ancora meglio. Quindi il machine learning aiuta come correttore del decoder classico, non come sostituto.

► CAMBIA SLIDE · 10

Risultato 2: Gross code e numero di qubit

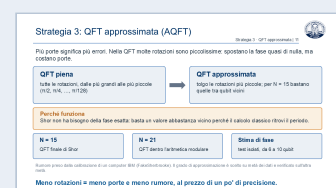


Infine il confronto tra i due codici. Per proteggere 12 qubit logici il Gross code, la barra arancione, usa 144 qubit di dato, da 10 a 14 volte meno del surface code.

Nel modello semplificato che ho usato, con un errore fisico dallo 0,75% in su, protegge anche meglio. E anche qui il decoder è decisivo: con la versione giusta gli errori scendono di circa cento volte.

► CAMBIA SLIDE · 11

Strategia 3: QFT approssimata (AQFT)



Terza strategia: la QFT approssimata. L'idea è semplice: gli errori vengono dalle porte, quindi meno porte significa meno errori.

Nella QFT ci sono rotazioni sempre più piccole, che spostano la fase quasi di nulla ma costano porte. Nella versione approssimata le tolgo; per N uguale a 15 bastano quelle tra qubit vicini. Funziona perché Shor non ha bisogno della fase

esatta: basta un valore abbastanza vicino perché il calcolo classico ritrovi il periodo.

L'ho provata su N uguale a 15, su N uguale a 21 e su test di stima di fase, con il rumore misurato su un computer IBM e riprodotto in simulazione.

► CAMBIA SLIDE · 12

Risultato 3: AQFT su N = 15



Ed ecco il risultato. A destra c'è la QFT piena, verso sinistra tolgo sempre più rotazioni. Con la QFT piena Shor si ferma al 48%; con il grado scelto, quello cerchiato, sale al 63%: quasi 15 punti in più, con il 23% di porte in meno.

Va detto però che 15 è un caso favorevole. Su N uguale a 21 le porte si dimezzano, ma il vantaggio non è dimostrato: l'approssimazione va scelta caso per caso.

► CAMBIA SLIDE · 13

Demo: Shor passo per passo, con e senza rumore



[Il video parte da solo e dura 2 minuti. Parla sopra il video, seguendo le scritte in basso; i tempi tra parentesi sono quelli del video.]

[0:00] Prima delle conclusioni, vi mostro il simulatore che ho realizzato: segue Shor passo per passo, prima senza errori e poi con il rumore.

[0:06] Il caso è quello della tesi: N uguale a 15, base 7. Ogni riga è un qubit, e a destra c'è la sua sfera di Bloch, che mostra lo stato del qubit con una freccia.

[0:17] Le porte di Hadamard mettono i qubit in sovrapposizione. Poi le moltiplicazioni per 7 modulo 15 scrivono il periodo nelle fasi; le frecce si accorciano, ma qui non è un errore: è entanglement.

[0:30] La QFT inversa trasforma il periodo in quattro picchi, e alla misura il calcolo classico ricava 15 uguale 3 per 5.

[0:48] Ora lo stesso circuito con il rumore. Le frecce che prima restavano intere si accorciano: lo stato perde coerenza.

[1:08] Qui si vede dove entra il rumore: ogni porta ha il suo errore, le porte a due qubit sono le più fragili e la misura può sbagliare il bit.

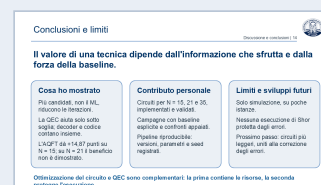
[1:15] Ripetendo il circuito molte volte, le misure si spargono su tutti gli esiti e i picchi si abbassano: è lo stesso calo che avete visto nel grafico del rumore.

[1:33] Questa è informazione persa, che dopo non si recupera.

[1:53] Ed è il messaggio della tesi: il rumore va affrontato durante il calcolo, perché dopo la misura l'informazione distrutta non si ricostruisce.

► CAMBIA SLIDE · 14

Conclusioni e limiti



Per concludere: una tecnica vale quanto l'informazione che riesce a sfruttare, e va sempre confrontata con un riferimento adeguato.

Nel post-processing il vantaggio viene dal provare più candidati, non dal machine learning. La correzione degli errori funziona solo sotto soglia, e codice e decoder contano insieme. La QFT approssimata aiuta, ma non sempre: su N uguale a 15 sì, su 21 non è dimostrato.

Il mio contributo è stato costruire e verificare i circuiti, progettare confronti corretti e rendere tutto riproducibile.

I limiti sono chiari: ho lavorato in simulazione, su pochi casi, e senza un'esecuzione di Shor protetta dagli errori. Il passo successivo è rendere i circuiti più leggeri e unirli alla correzione degli errori.

► CAMBIA SLIDE · 15

Grazie per l'attenzione



Vi ringrazio per l'attenzione. Sono a disposizione per le vostre domande.